

작가와 독자가 함께 만드는
이야기 공간, 라이트북스

라이트북스 담당 기능 개발 내역 - 이지해

LIGHIT BOOKS



팀장	이지해
팀원	최호철, 허유진
프로젝트 기간	2025-03-10 ~ 2025-04-02
깃 허브 주소	https://github.com/EZHae/lightbooks-project

라잇북스 담당 기능 개발 내역 목차

- 1 ——— 사용자 페이지
 - └ 회원가입, 로그인, 알림, 결제 및 사용 내역
- 2 ——— 어드민 페이지
 - └ 공지사항, 결제 대기
- 3 ——— 작품 관리 페이지
 - └ 작품 별 수익 관리
- 4 ——— 결제 시스템
 - └ 코인 충전 및 사용, 마일리지 적립, 이용권 교환, 작품 구매, 작품 후원
- 5 ——— 실시간 채팅 시스템
 - └ 사용자 채팅, 어드민 채팅
- 6 ——— 데이터베이스(ERD)

로그인 소셜 도서 제목, 작가를 입력하세요.

아이디

비밀번호

닉네임

이름

성별 ☒ 남성 ☐ 여성 태어난 연도 2025

전화번호

이메일

프로필

가입

```

9  @Data
10 public class UserSignUpDto {
11
12     private String loginId;
13     private String password;
14     private String nickname;
15     private String username;
16     private Integer gender;
17     private Integer year;
18     private String phonenumber;
19     private String email;
20     private String imgSrc;
21
22     public User toEntity>PasswordEncoder passwordEncoder) {
23         return User.builder()
24             .loginId(loginId)
25             .password(passwordEncoder.encode(password))
26             .nickname(nickname)
27             .username(username)
28             .gender(gender)
29             .year(year)
30             .phonenumber(phonenumber)
31             .email(email)
32             .imgSrc(imgSrc)
33             .build();
34     }
35 }

```

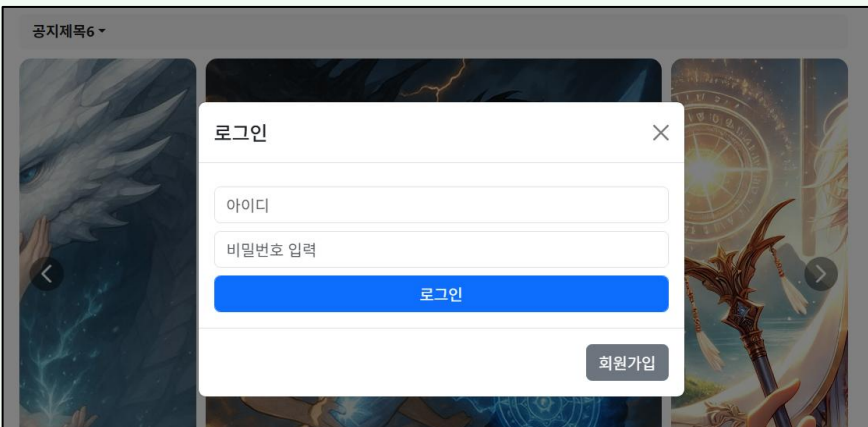
```

// 비밀번호를 암호화하는 객체를 스프링 컨테이너가 빈
@Bean // -> 스프링 컨테이너에서 관리하는 객체(빈) ->
PasswordEncoder passwordEncoder() {
    log.info("BCryptPasswordEncoder 생성");
    return new BCryptPasswordEncoder();
}

```

	id	login_id	password	
	5	ezhae1	\$2a\$10\$U.3BUJLgtAkbVtniqnb7meHKMvp3GVPd...	0
	6	choiho013	\$2a\$10\$Z2jA1i6pFoB9FZCYrBWJKeWhp8Wp0b...	초
	7	yudi	\$2a\$10\$haT6KYlO9ewcYedCYSSzwwVqMia7dTa	드

- 사용자가 회원가입 시 입력한 정보를 DB에 저장.
- 비밀번호의 경우 암호화하여 저장.



```
// 로그인 성공 핸들러: 로그인 요청을 보낸 페이지로 이동
private void handleSuccess(HttpServletRequest request, HttpServletResponse response,
    Authentication authentication) throws IOException {
    log.info("로그인 성공: {}", authentication.getName());

    request.getSession().removeAttribute("errorMessage");

    // 이전 페이지(로그인 요청을 보낸 페이지) URL 가져오기
    String referer = request.getHeader("Referer");
    if (referer == null || referer.isEmpty()) {
        referer = "/"; // Referer가 없으면 홈으로 이동
    }

    // 로그인 성공 후 이전 페이지로 리다이렉트
    response.sendRedirect(referer);
}
```

```
http.formLogin(t -> t.loginProcessingUrl("/user/signin") // 로그인 요청을 처리할 URL
    .successHandler(this::handleSuccess).failureHandler(this::handleFailure));

// 로그아웃 설정 (현재 페이지 유지)
http.logout(t -> t.logoutUrl("/user/signout") // 로그아웃 요청 URL
    .logoutSuccessHandler(this::handleLogout) // 로그아웃 후 현재 페이지로 이동
    .invalidateHttpSession(true) // 세션 무효화
    .deleteCookies("JSESSIONID") // JSESSIONID 쿠키 삭제
);
```

```
// 로그인 실패 핸들러: 홈("/")으로 이동하며 오류 메시지 전달
private void handleFailure(HttpServletRequest request, HttpServletResponse response,
    AuthenticationException exception) throws IOException {
    log.info("로그인 실패: {}", exception.getMessage());

    // 이전 페이지(로그인 요청을 보낸 페이지) URL 가져오기
    String referer = request.getHeader("Referer");
    if (referer == null || referer.isEmpty()) {
        referer = "/"; // Referer가 없으면 홈으로 이동
    }

    // 세션에 에러 메시지 저장
    request.getSession().setAttribute("errorMessage", "아이디 또는 비밀번호를 확인하세요.");

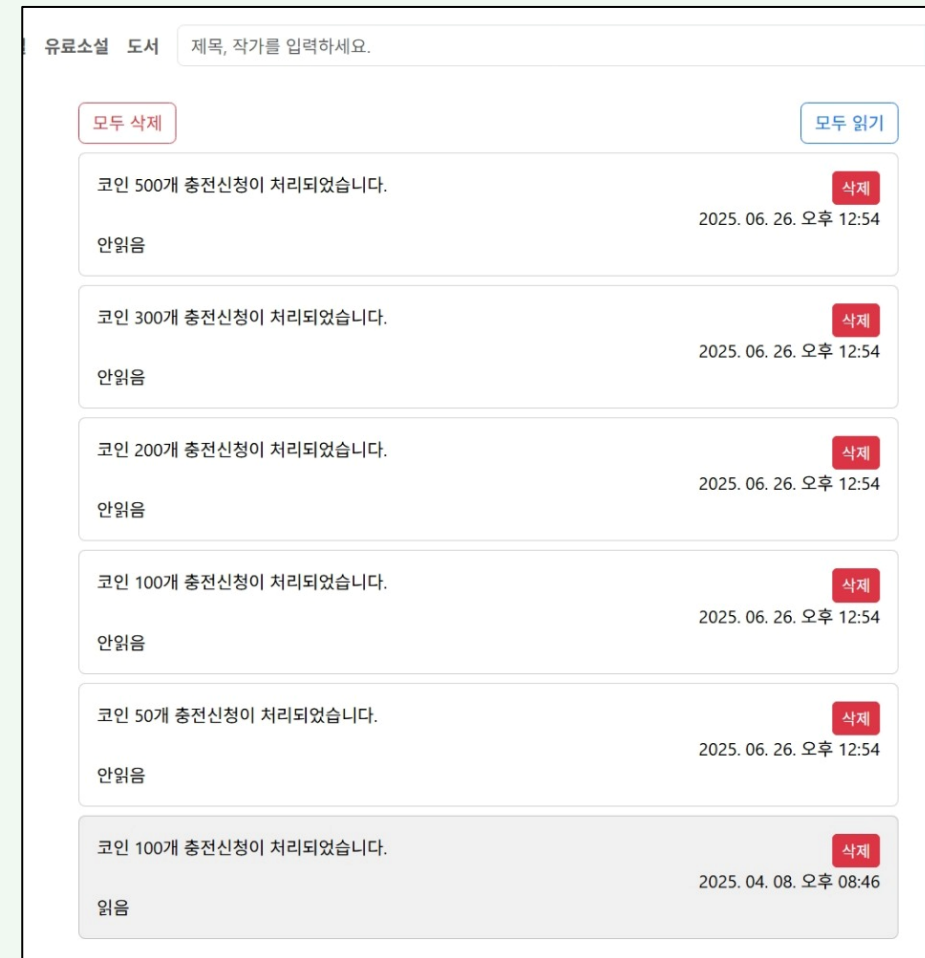
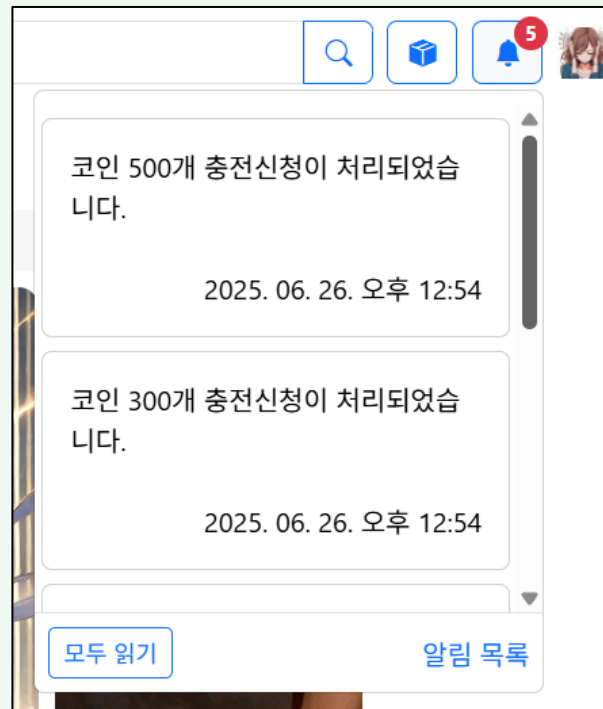
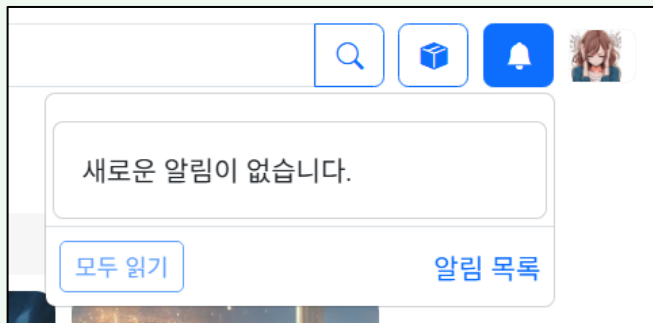
    // 로그인 실패 후 이전 페이지로 리다이렉트
    response.sendRedirect(referer);
}
```

- 사용자가 로그인 정보를 틀리게 입력하였을 경우, 사용자에게 로그인 실패를 알리기 위하여 세션에 에러 메시지를 저장한 후 해당 메시지를 사용자에게 출력.
- 사용자가 로그인에 성공하였을 경우 저장되어 있는 에러 메시지 세션을 삭제한 후 이전 페이지로 리다이렉트.

```
33 @EqualsAndHashCode(onlyExplicitlyIncluded = true, callSuper = false)
34 @Entity
35 @Table(name = "USERS")
36 public class User extends BaseTimeEntity implements UserDetails {
37
38     @Id
39     @GeneratedValue(strategy = GenerationType.IDENTITY)
40     private Long id;
41
42     @Column(name = "login_id")
43     @EqualsAndHashCode.Include
44     private String loginId;
45     private String password;
46     private String nickname;
47     private String username;
48     private Integer gender;
49     private Integer year;
50     private String phoneNumber;
51     private String email;
52
53     @Column(name = "access_time")
54     private LocalDateTime accessTime;
55
56     @Column(name = "img_src")
57     private String imgSrc;
58
59     @Column(name = "today_check", nullable = false, insertable = false)
60     private Integer todayCheck;
61
62     @JsonIgnore
63     @ToString.Exclude
64     @OneToOne(mappedBy = "user", fetch = FetchType.LAZY, cascade = CascadeType.ALL, orphanRemoval = true)
65     private UserWallet userWallet;
66
67     @Override
68     public Collection<? extends GrantedAuthority> getAuthorities() {
69         // TODO Auto-generated method stub
70         return null;
71     }
}
```

```
41 @Service
42 public class UserService implements UserDetailsService {
43
44     private final UserRepository userRepo;
45     private final MileagePaymentRepository mileagePaymentRepo;
46     private final TicketRepository ticketRepo;
47     private final PasswordEncoder passwordEncoder;
48     private final NovelRepository novelRepo;
49     private final NotificationRepository notiRepo;
50     private final CoinPaymentRepository coinPaymentRepo;
51
52     @Override
53     public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
54         String loginId = username;
55         log.info("loginId={}", loginId);
56
57         Optional<User> user = userRepo.findByLoginId(loginId);
58         log.info("{} ", user);
59         if (user.isPresent()) {
60             User loginUser = user.get();
61
62             int todayCheck = loginUser.getTodayCheck();
63             if (todayCheck == 0) {
64                 updateCheck(loginUser);
65             }
66
67             return user.get();
68         } else {
69             throw new UsernameNotFoundException(loginId + "과 일치하는 사용자 없음");
70         }
71     }
}
```

- User 엔티티와 서비스에서 각각 UserDetails, UserDetailsService를 상속받아 로그인 방식을 구현.
- 로그인 시 오늘의 출석체크 유무를 확인하여 출석체크가 안되어 있을 경우 해당 값을 변경하여 저장.
해당 값을 변경하면서 마일리지를 적립. (이용권 교환에 사용)



- 코인 충전 신청 처리, 작품 등급 전환 신청처리, 작품후원(작가인 경우)에 대한 알림을 출력.
- 새 알림이 없거나, 삭제할 알림이 없는 경우 해당 버튼 비활성화.


```
1 CREATE DEFINER='ezhae'@'%' TRIGGER `trg_coin_payment_update` AFTER UPDATE ON `COIN_PAYMENT_WAITING`
2 DECLARE notification_msg VARCHAR(255);
3 DECLARE notification_target VARCHAR(255);
4
5 -- con 값이 1로 변경될 때만 실행
6 IF OLD.con = 0 AND NEW.con = 1 THEN
7     -- 알림에 따라 다른 메시지 설정
8     CASE NEW.type
9         WHEN 0 THEN
10             SET notification_msg = CONCAT('코인 ', ABS(NEW.coin), '개 충전신청이 처리되었습니다.');
```

코인 충전 신청 처리 결과 알림

```
29 ELSEIF NEW.type = 3 THEN
30     -- 후원 시 작가 조회
31     SELECT user_id, title INTO author_id, novel_title
32     FROM NOVELS
33     WHERE id = NEW.novel_id;
34
35     SELECT nickname INTO donor_nickname
36     FROM USERS
37     WHERE id = NEW.user_id;
38
39     SET notification_msg = CONCAT(
40         '...', IFNULL(novel_title, '알 수 없는'), '...', '작품에 ',
41         '...', IFNULL(donor_nickname, '작품'), '...', '님이 코인 ',
42         ABS(NEW.coin), '개를 후원하였습니다.'
43     );
44     SET notification_target = CONCAT('/myworks/income/details?id=', author_id, '&novelId=', NEW.novel_id);
45
46     INSERT INTO NOTIFICATIONS (user_id, msg, is_read, target, created_time)
47     VALUES (
48         author_id, -- 작가의 user_id
49         notification_msg,
50         0, -- 읽음 여부 (0: 읽지 않음)
51         notification_target, -- 후원받은 작품 ID를 target으로 지정
52         NOW()
53     );
```

작품 후원 알림

- 어드민이 신청을 처리하였을 경우(코인 신청 대기 데이터의 con 값이 0에서 1로 변경될 경우) 해당 사용자에게 알림을 생성

- 다른 사용자가 자신의 작품에 후원하였을 경우(type 값이 3인 코인 사용 내역 데이터가 추가되었을 경우) 작품의 작가에게 알림을 생성

id	user_id	msg	is_read	target	creat
92	5	코인 100개 충전신청이 처리되었습니다.	0	/user/coinpayment?id=5	2025-
91	5	코인 50개 충전신청이 처리되었습니다.	0	/user/coinpayment?id=5	2025-
90	5	코인 100개 충전신청이 처리되었습니다.	1	/user/coinpayment?id=5	2025-
87	6	"붉은 여왕" 작품에 "이지해99"님이 코인 20...	1	/myworks/income/details?id=6&novelId=...	2025-
86	6	"붉은 여왕" 작품에 "도도"님이 코인 444개를 ...	1	/myworks/income/details?id=6&novelId=...	2025-

코인 50개 충전신청이 처리되었습니다.

삭제

안읽음
2025. 06. 26. 오후 12:54

코인 100개 충전신청이 처리되었습니다.

삭제

읽음
2025. 04. 08. 오후 08:46

```

18 @Modifying
19 @Transactional
20 @Query("UPDATE Notification n SET n.isRead = 1 WHERE n.userId = :userId AND n.isRead = 0")
21 void updateNotificationsAllReadByUserId(Long userId);
28 @Modifying
29 @Transactional
30 @Query("UPDATE Notification n SET n.isRead = 1 WHERE n.id = :id")
31 void updateNotificationReadById(Long id);

```

- 알림 클릭 시 DB에 저장되어 있는 target 컬럼의 속성 값으로 주소 이동.
- 추가로 알림 클릭 혹은 모두 읽기 버튼 클릭 시 상황에 맞는 쿼리문을 호출하여 is_read 속성 값을 변경하여 저장.

라잇

추천 무료소설 유료소설 도서 제목, 작가를 입력하세요.

🔍

🏠

🔔

👤

코인 충전 내역

구분	충전 코인	사용 금액	충전 일시
코인 충전	100	10000	2025. 04. 01. 오후 06:11
코인 충전	100	10000	2025. 04. 01. 오후 05:06
코인 충전	50	5000	2025. 03. 31. 오후 09:07
코인 충전	100	10000	2025. 03. 30. 오후 09:57
코인 충전	1000	100000	2025. 03. 26. 오후 07:37

이전

1

2

3

4

5

다음

작품 구매 내역

구분	작품명	회차	사용 코인	구매 일시
작품 구매	호트러진 차원	3	5	2025. 04. 02. 오전 07:40
작품 구매	휘날리는 빛꽃 아래	3	5	2025. 04. 02. 오전 07:39
작품 구매	일단 돈부터 주세요!	6	5	2025. 04. 02. 오전 07:26
작품 구매	일단 돈부터 주세요!	4	5	2025. 04. 02. 오전 07:25
작품 구매	일단 돈부터 주세요!	4	5	2025. 04. 02. 오전 07:25

이전

1

2

다음

코인 환불/환전 내역

환불/환전 신청

구분	환불/환전 코인	환불/환전 금액	환불 일시	결과
환불/환전	420	37800	2025. 03. 26. 오전 01:27	완료
환불/환전	200	18000	2025. 03. 26. 오전 01:21	완료
환불/환전	100	9000	2025. 03. 19. 오후 05:13	완료
환불/환전	200	18000	2025. 03. 19. 오후 04:24	완료
환불/환전	100	9000	2025. 03. 19. 오후 04:24	완료

이전

1

2

다음

후원 내역

구분	작품명	후원 코인	후원 일시
작품 후원	1학년 3반의 전쟁	135	2025. 04. 01. 오후 04:49

이전

1

다음

라잇	추천	무료소설	유료소설	도서	제목, 작가를 입력하세요.	🔍	🏠	🔔	👤								
이용권 보유 내역																	
이용권 종류	사용 가능 작품	보유 일시															
전체 이용권	-	2025. 03. 24. 오후 11:18															
전체 이용권	-	2025. 03. 24. 오후 11:17															
작품 이용권	작품이 삭제되어 전체 이용권으로 사용 가능한 이용권	2025. 03. 20. 오후 08:05															
작품 이용권	작품이 삭제되어 전체 이용권으로 사용 가능한 이용권	2025. 03. 20. 오후 06:53															
이전 1 2 3 다음																	
이용권 사용 내역																	
사용 작품	사용 회차	사용 일시															
일단 돈부터 주세요!	2	2025. 04. 01. 오후 04:19															
이전 1 다음																	
라잇	추천	무료소설	유료소설	도서	제목, 작가를 입력하세요.	🔍	🏠	🔔	👤								
마일리지 적립 내역																	
구분	적립 마일리지	설명	적립 일시														
적립	100	출석 적립	2025. 04. 02. 오전 03:43														
적립	100	출석 적립	2025. 04. 01. 오전 10:35														
적립	100	출석 적립	2025. 03. 29. 오후 09:22														
적립	100	출석 적립	2025. 03. 29. 오후 08:01														
적립	100	출석 적립	2025. 03. 27. 오후 09:01														
이전 1 2 다음																	
마일리지 사용 내역																	
구분	사용 마일리지	설명	사용 일시														
사용	100	이용권 교환	2025. 03. 26. 오전 11:58														
사용	50	이용권 교환	2025. 03. 20. 오후 08:04														
사용	100	이용권 교환	2025. 03. 20. 오후 08:04														
사용	50	이용권 교환	2025. 03. 20. 오후 06:53														
사용	100	이용권 교환	2025. 03. 20. 오후 06:53														
이전 1 2 다음																	

- 사용자의 코인 충전/환불, 작품 구매/후원, 이용권 보유/사용, 마일리지 적립/사용 내역을 출력.

	id	user_id	type	novel_id	novel_title	episode_id	episode_num	created_time	coin	cash	donation_msg
	227	5	1	164	휘날리는 벚꽃 아래	617	3	2025-04-02 07:39:59	-5	NULL	NULL
	228	5	1	152	호트러진 차원	523	3	2025-04-02 07:40:44	-5	NULL	NULL
	229	5	0	NULL	NULL	NULL	NULL	2025-04-02 11:03:28	50	5000	NULL
	230	5	2	NULL	NULL	NULL	NULL	2025-04-02 12:05:24	-100	9000	NULL
	231	32	3	119	붉은 여왕	NULL	NULL	2025-04-02 14:02:32	-1000	NULL	NULL
	232	35	3	119	붉은 여왕	NULL	NULL	2025-04-02 14:03:09	-1001	NULL	NULL

코인 사용 내역 type 값	설명
0	코인 충전
1	작품 구매
2	코인 환불/환전
3	작품 후원

- 사용자가 코인을 충전/사용할 경우 상황에 맞는 데이터를 저장.

	id	user_id	type	mileage	descrip	created_time
	14	17	0	100	출석 적립	2025-03-20 01:46:02
	15	5	0	100	출석 적립	2025-03-20 06:49:12
	16	5	1	-50	이용권 교환	2025-03-20 18:45:15
	17	5	1	-100	이용권 교환	2025-03-20 18:45:30

마일리지 사용 내역 type 값	설명
0	마일리지 적립
1	마일리지 사용(이용권 교환)

- 사용자의 마일리지 적립/사용될 경우 상황에 맞는 데이터를 저장.

	id	user_id	grade	novel_id	created_time
	24	5	1	NULL	2025-03-20 18:53:52
	28	5	1	NULL	2025-03-20 20:05:01
	101	5	0	NULL	2025-03-24 23:19:36
	159	5	0	NULL	2025-04-02 15:19:36
	161	5	1	153	2025-06-26 14:27:07

티켓 보유 내역 grade 값	설명
0	전체 작품 이용권
1	특정 작품 이용권 (작품이 삭제될 경우 novel_id 값이 null로 변경되며 전체 작품 이용권의 역할을 할 수 있음)

※ 티켓 사용 내역

	id	ticket_id	user_id	novel_id	novel_title	episode_id	episode_num	created_time
	88	98	5	125	일단 돈부터 주세요!	299	2	2025-04-01 16:19:22
	92	129	6	153	이끌림의 끝	541	1	2025-04-01 18:49:39
	93	152	33	164	휘날리는 벚꽃 아래	615	1	2025-04-02 11:47:44
	94	99	5	153	이끌림의 끝	542	2	2025-04-02 15:17:43
	95	160	5	153	이끌림의 끝	545	5	2025-06-26 14:25:25
	96	100	5	153	이끌림의 끝	548	6	2025-06-26 14:25:38

id	user_id	msg	is_read	target	creat
92	5	코인 100개 충전신청이 처리되었습니다.	0	/user/coinpayment?id=5	2025-
91	5	코인 50개 충전신청이 처리되었습니다.	0	/user/coinpayment?id=5	2025-
90	5	코인 100개 충전신청이 처리되었습니다.	1	/user/coinpayment?id=5	2025-
87	6	"붉은 여왕" 작품에 "이지해99"님이 코인 20...	1	/myworks/income/details?id=6&novelId=...	2025-
86	6	"붉은 여왕" 작품에 "도도"님이 코인 444개를 ...	1	/myworks/income/details?id=6&novelId=...	2025-

코인 50개 충전신청이 처리되었습니다.

삭제

안읽음
2025. 06. 26. 오후 12:54

코인 100개 충전신청이 처리되었습니다.

삭제

읽음
2025. 04. 08. 오후 08:46

```

18 @Modifying
19 @Transactional
20 @Query("UPDATE Notification n SET n.isRead = 1 WHERE n.userId = :userId AND n.isRead = 0")
21 void updateNotificationsAllReadByUserId(Long userId);

28 @Modifying
29 @Transactional
30 @Query("UPDATE Notification n SET n.isRead = 1 WHERE n.id = :id")
31 void updateNotificationReadById(Long id);

```

- 알림 클릭 시 DB에 저장되어 있는 target 컬럼의 속성 값으로 주소 이동.
- 추가로 알림 클릭 혹은 모두 읽기 버튼 클릭 시 상황에 맞는 쿼리문을 호출하여 is_read 속성 값을 변경하여 저장.

소설 유료소설 도서 어드민메뉴 ▾

제목, 작가를 입력하세요.

을 라잇 베스트 오늘신작

공지제목6 ▾

공지제목6
공지제목3
공지제목15
공지제목7
공지제목12

공지사항			
강조	제목	작성자	작성일
☒	공지제목6	어드민	2025. 03. 24. 오후 01:24
☒	공지제목3	어드민	2025. 03. 24. 오후 01:24
☒	공지제목15	어드민	2025. 03. 24. 오후 05:31
☒	공지제목7	어드민	2025. 03. 24. 오후 01:24
☒	공지제목12	어드민	2025. 03. 24. 오후 03:31
☐	이거슨 공지다	어드민	2025. 03. 26. 오전 12:43
☐	이거시 공지다	어드민	2025. 03. 24. 오후 06:20
☐	공지제목11	어드민	2025. 03. 24. 오후 03:31
☐	공지제목10	어드민	2025. 03. 24. 오후 03:08
☐	공지제목9	어드민	2025. 03. 24. 오후 03:05
☐	공지제목4	어드민	2025. 03. 24. 오후 01:24

- 어드민 계정에서 작성된 공지사항 중 어드민이 강조 표시를 한 공지사항을 메인 홈페이지에 출력. (최대 5개)

공지사항		
강조	제목	작성자
☒	공지제목6	어드민
☒	공지제목3	어드민
☒	공지제목15	어드민
☒	공지제목7	어드민
☒	공지제목12	어드민
☐	이거슨 공지다	어드민
☐	이거시 공지다	어드민
☐	공지제목11	어드민
☐	공지제목10	어드민
☐	공지제목9	어드민
☐	공지제목4	어드민

```

37 // highlight가 1인 항목과 0인 항목을 분리해서 각각 정렬
38 let highlightedPosts = noticeData.filter(post => post.highlight === 1);
39 let regularPosts = noticeData.filter(post => post.highlight === 0);
40
41 // 1. highlight가 1인 항목은 modifiedTime 기준으로 내림차순 정렬
42 highlightedPosts.sort((a, b) => new Date(a.modifiedTime) - new Date(b.modifiedTime));
43
44 // 2. highlight가 0인 항목은 createdTime 기준으로 오름차순 정렬
45 regularPosts.sort((a, b) => new Date(b.createdTime) - new Date(a.createdTime));
46
47 // 두 배열을 합치기
48 const sortedPosts = [...highlightedPosts, ...regularPosts];
49
50 // 정렬된 데이터를 HTML로 표시
51 if (sortedPosts.length > 0 && userId == 16) {
52   sortedPosts.forEach(post => {
53     let formatCreatedTime = formatDate(post.createdTime);
54     html +=
55       `
56       <tr class="row">
57         <td class="col-1" ${post.highlight === 1 ? 'style="background: #f0f0f0"' : ''}>
58           <div class="form-check form-switch">
59             <input class="form-check-input highlight-switch" type="checkbox" role="switch" data-id="${post.id}" ${post.highlight === 1
60             </div>
61           </td>
62           <td class="col-6" ${post.highlight === 1 ? 'style="background: #f0f0f0"' : ''}>
63             <a class="text-black link-offset-2 link-offset-3-hover link-underline-dark link-underline-opacity-0 link-underline-opacity-75-t
64               ${post.highlight === 1 ? '<strong>${post.title}</strong>' : post.title}
65             </a>
66           </td>
67           <td class="col-2" ${post.highlight === 1 ? 'style="background: #f0f0f0"' : ''}>${post.writer}</td>
68           <td class="col-3" ${post.highlight === 1 ? 'style="background: #f0f0f0"' : ''}>${formatCreatedTime}</td>
69         </tr>
70       `;
71   });
72   noticeTBody.innerHTML = html;

```

- DB에 저장되어 있는 공지사항 중 highlight 컬럼 값이 1인 경우를 먼저 출력 후, 나머지를 출력.

공지사항			
강조	제목	작성자	작성일
☒	공지제목6	어드민	2025. 03. 24. 오후 01:24
☒	공지제목3	어드민	2025. 03. 24. 오후 01:24
☒	공지제목15	어드민	2025. 03. 24. 오후 05:31
☒	공지제목7	어드민	2025. 03. 24. 오후 01:24
☒	공지제목12	어드민	2025. 03. 24. 오후 03:31
☐	이거슨 공지다	어드민	2025. 03. 26. 오전 12:43
☐	이거시 공지다	어드민	2025. 03. 24. 오후 06:20
☐	공지제목11	어드민	2025. 03. 24. 오후 03:31
☐	공지제목10	어드민	2025. 03. 24. 오후 03:08
☐	공지제목9	어드민	2025. 03. 24. 오후 03:05
☐	공지제목4	어드민	2025. 03. 24. 오후 01:24

```
94 // 스위치 이벤트 리스너 추가
95 function addSwitchEventListeners(noticeData) {
96   const switches = document.querySelectorAll('.highlight-switch');
97   const currentHighlightCount = noticeData.filter(post => post.highlight === 1).length;
98
99   switches.forEach(switchElem => {
100     switchElem.addEventListener('change', (e) => {
101       const postId = e.target.getAttribute('data-id');
102       const newHighlight = e.target.checked ? 1 : 0;
103
104       // highlight가 1로 설정될 수 있는 최대 개수 확인
105       if (newHighlight === 1 && currentHighlightCount >= 5) {
106         alert('최대 5개의 공지만 강조할 수 있습니다.');
```

- 스위치를 할당하여 어드민이 웹 페이지에서 highlight 컬럼의 값을 변경 가능.
- 어드민 계정으로 로그인되어 있지 않은 경우 웹페이지의 '강조' 컬럼은 미출력.

라잇 추천 무료소셜 유료소셜 도서 어드민메뉴 ▾ 제목, 작가를 입력하세요.								
전체 보기 처리 완료 처리 대기								
아이디	유저아이디	유저이름	구분	코인	현금	신청일	상태	결제확인
62	ezhae1	이지해	코인 충전	1000	100000	2025. 06. 26. 오후 12:54	대기 중	<button>확인</button>
61	ezhae1	이지해	코인 충전	500	50000	2025. 06. 26. 오후 12:53	완료	완료
60	ezhae1	이지해	코인 충전	300	30000	2025. 06. 26. 오후 12:53	완료	완료
59	ezhae1	이지해	코인 충전	200	20000	2025. 06. 26. 오후 12:53	완료	완료
58	ezhae1	이지해	코인 충전	100	10000	2025. 06. 26. 오후 12:53	완료	완료
57	ezhae1	이지해	코인 충전	50	5000	2025. 06. 26. 오후 12:53	대기 중	<button>확인</button>
56	ezhae1	이지해	코인 충전	50	5000	2025. 06. 26. 오후 12:53	완료	완료
55	ezhae1	이지해	코인 충전	100	10000	2025. 04. 08. 오후 08:40	완료	완료
54	ezhae1	이지해	코인 충전	50	5000	2025. 04. 08. 오후 08:40	대기 중	<button>확인</button>
53	ezhae1	이지해	코인 충전	300	30000	2025. 04. 02. 오후 03:22	완료	완료
52	ezhae1	이지해	코인 환불/환전	-100	9000	2025. 04. 02. 오후 12:05	대기 중	<button>확인</button>

	id	user_id	type	coin	cash	created_time	con
	52	5	2	-100	9000	2025-04-02 12:05:24	0
	53	5	0	300	30000	2025-04-02 15:22:24	1
	54	5	0	50	5000	2025-04-08 20:40:42	0
	55	5	0	100	10000	2025-04-08 20:40:50	1
	56	5	0	50	5000	2025-06-26 12:53:32	1
	57	5	0	50	5000	2025-06-26 12:53:39	0
	58	5	0	100	10000	2025-06-26 12:53:43	1
	59	5	0	200	20000	2025-06-26 12:53:47	1
	60	5	0	300	30000	2025-06-26 12:53:52	1
	61	5	0	500	50000	2025-06-26 12:53:56	1
	62	5	0	1000	100000	2025-06-26 12:54:00	0

코인 결제 대기 내역 type 값	설명
0	코인 충전 신청
2	코인 환불/환전 신청

코인 결제 대기 내역 con 값	설명
0	처리 대기
1	처리 완료

- 사용자가 코인 충전/환불/환전 한 경우 COIN_PAYMENT_WATING 테이블에 데이터 생성.
- 결제 확인 버튼 클릭 시 con 값을 1로 변경하여 저장.

무료소설 유료소설 도서 어드민메뉴 ▼ 제목, 작가를 입력하세요.

리 완료 처리 대기

아이디	유저이름	구분	코인	현금	신청일	상태	결제확인
ee1	이지해	코인 충전	1000	100000	2025. 06. 26. 오후 12:54	대기 중	<button>확인</button>
ee1	이지해	코인 충전	500	50000	2025. 06. 26. 오후 12:53	완료	완료
ee1	이지해	코인 충전	300	30000	2025. 06. 26. 오후 12:53	완료	완료
ee1	이지해	코인 충전	200	20000	2025. 06. 26. 오후 12:53	완료	완료
ee1	이지해	코인 충전	100	10000	2025. 06. 26. 오후 12:53	완료	완료
ee1	이지해	코인 충전	50	5000	2025. 06. 26. 오후 12:53	대기 중	<button>확인</button>
ee1	이지해	코인 충전	50	5000	2025. 06. 26. 오후 12:53	완료	완료
ee1	이지해	코인 충전	100	10000	2025. 04. 08. 오후 08:40	완료	완료
ee1	이지해	코인 충전	50	5000	2025. 04. 08. 오후 08:40	대기 중	<button>확인</button>
ee1	이지해	코인 충전	300	30000	2025. 04. 02. 오후 03:22	완료	완료
ee1	이지해	코인 환불/환전	-100	9000	2025. 04. 02. 오후 12:05	대기 중	<button>확인</button>

```
1 CREATE DEFINER='ezhae'@'%' TRIGGER `trg_coin_payment_update` AFTER UPDATE ON `COIN_PAYMENT_WAITING` FOR EACH ROW BEGIN
2     DECLARE notification_msg VARCHAR(255);
3     DECLARE notification_target VARCHAR(255);
4
5     -- con 값이 1로 변경될 때만 실행
6     IF OLD.con = 0 AND NEW.con = 1 THEN
7         -- 알림에 따라 다른 메시지 설정
8         CASE NEW.type
9             WHEN 0 THEN
10             SET notification_msg = CONCAT('코인 ', ABS(NEW.coin), '개 충전신청이 처리되었습니다.');
```

- 결제 확인 버튼 클릭 시 처리 상태가 완료로 변경(con 값을 0에서 1로 변경)되며 해당 사용자의 처리 상태 변경 알림 데이터를 생성.

추천 무료소설 유료소설 도서

제목, 작가를 입력하세요.

내 작품 작품 작성



유료
일단 돈부터 주세요!

누적 판매금액
444 코인 [자세히보기](#)

누적 후원금액
450 코인 [자세히보기](#)

이전 1 다음

© 2025 LightBooks. All Rights Reserved.
자가와 독자가 함께 만드는 이야기 공간 라이브스

```
13 // 소설의 수익모델 별 누계액 조회
14 @Query(value = "SELECT n.id, n.title, n.grade, n.cover_src AS coverSrc, " +
15     "SUM(CASE WHEN cpi.type = 4 THEN cpi.coin ELSE 0 END) AS type4Income, " +
16     "SUM(CASE WHEN cpi.type = 5 THEN cpi.coin ELSE 0 END) AS type5Income " +
17     "FROM NOVELS n " +
18     "JOIN COIN_PAYMENT_INCOME cpi ON n.id = cpi.novel_id " +
19     "WHERE n.user_id = :userId " +
20     "GROUP BY n.id, n.title, n.grade, n.cover_src " +
21     "ORDER BY n.id ASC",
22     countQuery = "SELECT COUNT(DISTINCT n.id) FROM NOVELS n " + // total count query (required for pageable)
23     "JOIN COIN_PAYMENT_INCOME cpi ON n.id = cpi.novel_id " +
24     "WHERE n.user_id = :userId",
25     nativeQuery = true)
26 Page<Object[]> findById(@Param("userId") Long userId, Pageable pageable);

36 let html = '';
37 contentData.forEach(item => {
38     const novelId = item[0];
39     const novelTitle = item[1];
40     const novelGrade = (item[2] == 0) ? '무료' : '유료';
41     const novelCoverSrc = item[3];
42     const type4Income = item[4];
43     const type5Income = item[5];
44     html +=
45     `
46     <div class="col">
47         <div class="card card-body shadow-sm card-item mx-2 mt-2 mb-4">
48             <div class="row">
49                 <div class="col-4">
50                     
51                 </div>
52                 <div class="col-8">
53                     <div>
54                         <span>${novelGrade}</span>
55                     </div>
56                     <div class="mb-2 border-bottom border-black">
57                         <span class="fs-4">${novelTitle}</span>
58                     </div>
59                     <div class="border-bottom border-dark-subtle">
60                         <span>누적 판매금액</span><br>
61                         <span>${type4Income}</span> 코인</div>
62                     <div>
63                         <span>누적 후원금액</span><br>
64                         <span>${type5Income}</span> 코인</div>
65                     </div>
66                 </div>
67             </div>

```

- 수익이 발생한 작품들의 누적 판매/후원 금액을 함께 출력.

추천 무료소설 유료소설 도서 제목, 작가를 입력하세요.

내 작품 작품 작성 수익 관리 정산

판매 수익 후원 수익

일단 돈부터 주세요! 판매 수익 코인은 회차 구매 코인의 60%입니다. (소수점 버림)

구매자	회차	수익 코인	구매 일시
이지해99	6	3 코인	2025. 04. 02. 오전 07:26
이지해99	4	3 코인	2025. 04. 02. 오전 07:25
이지해99	5	3 코인	2025. 04. 01. 오후 05:58
이지해99	3	3 코인	2025. 04. 01. 오후 04:44
최호철	4	3 코인	2025. 04. 01. 오후 04:04
(알 수 없음)	3	3 코인	2025. 04. 01. 오후 03:15
(알 수 없음)	2	3 코인	2025. 04. 01. 오후 03:15
엘프	3	3 코인	2025. 03. 31. 오후 10:35
최호철	1	60 코인	2025. 03. 31. 오전 01:39
유디	6	60 코인	2025. 03. 31. 오전 12:23

이전 1 2 다음

```
50 function readIncome(contentData) {
51   console.log(contentData);
52   const contentSection = document.querySelector('tbody#contentSection');
53
54   if (contentData.length === 0) {
55     contentSection.innerHTML =
56
57       <tr class="row">
58         <td class="col">수익발생없음</td>
59       </tr>
60   };
61   return;
62 }
63
64 let html = '';
65 contentData.forEach(item => {
66   let nickname = (item.nickname == null) ? '(알 수 없음)' : item.nickname;
67   let episodeNum = (item.episodeNum == null) ? '-' : item.episodeNum;
68   let createdTime = formatDateTime(item.createdTime);
69   html +=
70
71     <tr class="row">
72       <td class="col">${nickname}</td>
73       <td class="col-2">${episodeNum}</td>
74       <td class="col-2 text-end">${item.coin} 코인</td>
75       <td class="col-4 text-end">${createdTime}</td>
76     </tr>
77
78   );
79   contentSection.innerHTML = html;
80 }
81
```

- 수익이 발생한 해당 작품의 COIN_PAYMENT_INCOME 테이블 데이터를 출력.

#4-1 코인 충전 및 사용

결제 시스템

라잇

추천 무료소설 유료소설 도서 어드민메뉴

제목, 작가를 입력하세요.

🔍

📦

🔔

코인 구매

💰 50 코인	5,000원	💰 100 코인	10,000원
💰 200 코인	20,000원	💰 300 코인	30,000원
💰 500 코인	50,000원	💰 1,000 코인	100,000원

코인이란?



회차 구매

마음에 드는 회차를 구매하실 수 있습니다.
언제든 어디서든 구매하신 소장 회차를 읽어보세
요.



작가 후원

응원하고 싶은 작가들에게 코인을 후원하실 수
있습니다. 후원된 코인은 작가분들이 완전히
수익으로 받으실 수 있습니다.

© 2025 LightBooks. All Rights Reserved.

작가와 독자가 함께 만드는 이야기 공간, 라잇북스

상호명: (주)라잇북스 대표자: 이최허 사업자등록번호: 123-45-67890 통신판매업신고: 2025-서울강남-1234
주소: 서울특별시 강남구 삼성대로 123, 라잇타워 5층 대표번호: 02-1234-5678 이메일: contact@lightbooks.co.kr

[이용약관](#) · [개인정보처리방침](#) · [고객센터](#)

20,000원

💰 300 코인

결제 수단 선택

❌

💰 50 코인

5,000원



무통장 입금

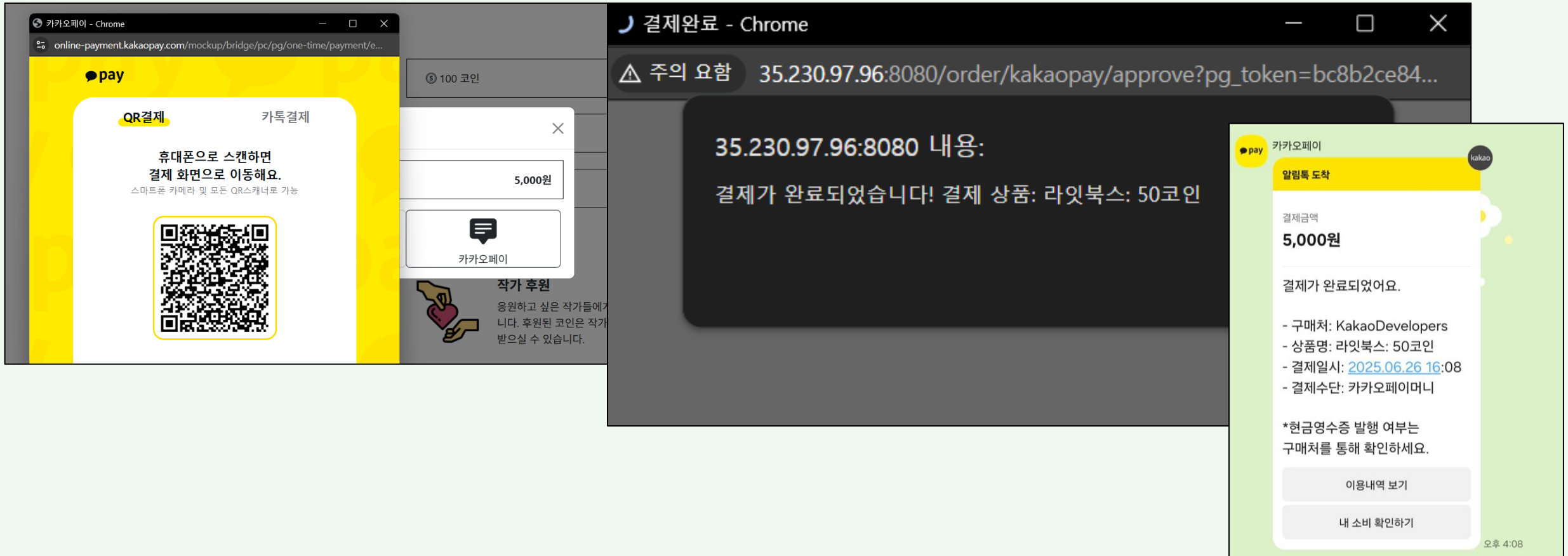


카카오페이

© 2025 LightBooks. All Rights Reserved.

작가와 독자가 함께 만드는 이야기 공간, 라잇북스

- 코인 충전 시 사용자는 결제 방법을 선택할 수 있음.
- 무통장 입금의 경우 17페이지의 어드민 - 결제 대기 [슬라이드 참조](#)



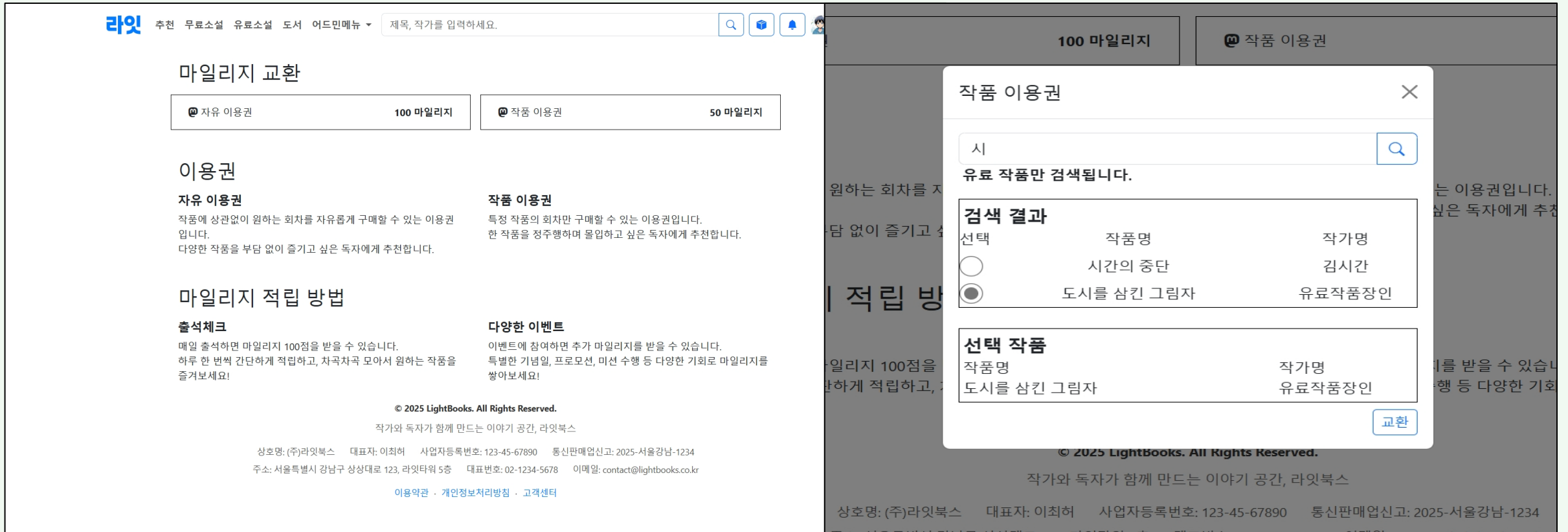
- 카카오페이 API를 활용하여 결제 환경 구현.
- 결제 완료 시 선택한 충전 코인만큼 보유 코인이 증가.

```
41 @Service
42 public class UserService implements UserDetailsService {
43
44     private final UserRepository userRepo;
45     private final MileagePaymentRepository mileagePaymentRepo;
46     private final TicketRepository ticketRepo;
47     private final PasswordEncoder passwordEncoder;
48     private final NovelRepository novelRepo;
49     private final NotificationRepository notifiRepo;
50     private final CoinPaymentRepository coinPaymentRepo;
51
52     @Override
53     public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
54         String loginId = username;
55         log.info("loginId={}", loginId);
56
57         Optional<User> user = userRepo.findByLoginId(loginId);
58         log.info("{} ", user);
59         if (user.isPresent()) {
60             User loginUser = user.get();
61
62             int todayCheck = loginUser.getTodayCheck();
63             if (todayCheck == 0) {
64                 updateCheck(loginUser);
65             }
66
67             return user.get();
68         } else {
69             throw new UsernameNotFoundException(loginId + "과 일치하는 사용자 없음");
70         }
71     }
72 }
```

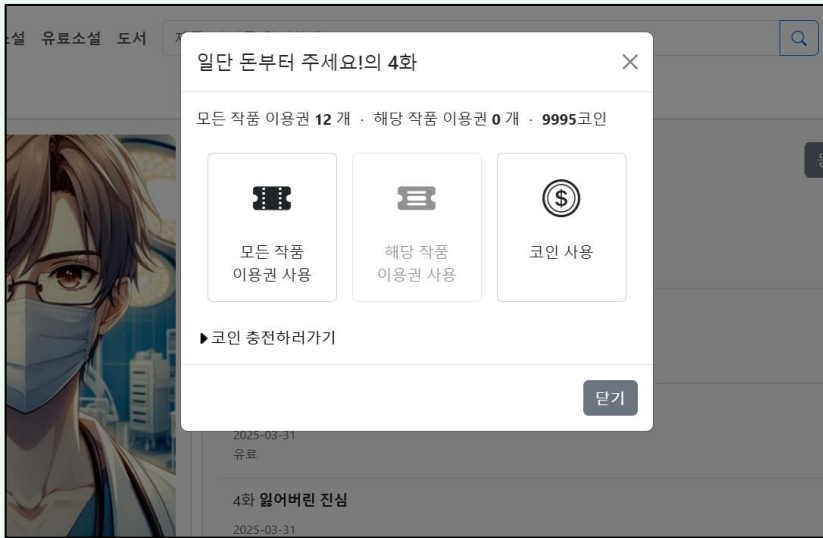
```
184 public void updateCheck(User user) {
185     user.updateTodayCheck(1);
186     userRepo.save(user);
187     MileagePayment mileagePayment = MileagePayment.builder()
188         .userId(user.getId())
189         .type(0)
190         .mileage(100L)
191         .descrip("출석 적립").build();
192     mileagePaymentRepo.save(mileagePayment);
193 }
```

	EVENT_CATALOG	EVENT_SCHEMA	EVENT_NAME	DEFINER	TIME_ZONE	EVENT_BODY	EVENT_DEFINITION	EVENT_TYPE	EXECUTE_AT	INTERVAL_VALUE	INTERVAL_FIELD	SC
▶	def	NOVELDB	reset_today_check	ezhae@%	SYSTEM	SQL	UPDATE USERS SET tod...	RECURRING	HULL	1	DAY	ON

- 사용자가 로그인할 때 출석체크가 되어 있지 않은 경우 today_check 값을 0에서 1로 변경한 후 마일리지를 적립하여 저장.
- 추가로 Mysql에서 이벤트 스케줄러를 생성하여 매일 자정이 되면 모든 유저의 today_check 값을 0으로 초기화.



- 적립한 마일리지를 이용권으로 교환하는 시스템.
- 자유 이용권의 경우 모든 유료 작품에 사용이 가능하며, 작품 이용권의 경우 유료 작품을 검색하여 해당 작품의 이용권으로 교환.



```

299 @ResponseBody
300 @PostMapping("/buy")
301 public ResponseEntity<?> episodeBuy(@RequestBody EpisodeBuyDto dto) {
302
303     int result = dto.getType();
304     Long ticketId;
305
306     if (result == 2) { // 타입이 2인 경우 (코인으로 구매)
307         orderService.saveCoinPaymentFromEpisodeBuyDto(dto); // 코인 이용내역 테이블 추가
308     } else { // 타입이 0, 1인 경우 (이용권으로 구매)
309         ticketId = ticketService.deleteTicketFromEpisodeBuyDto(dto); // 이용권 삭제, 서비스 메
310         orderService.saveTicketPaymentFromEpisodeBuyDto(dto, ticketId);
311     }
312     // 북마크 테이블 추가, 구매하면 북마크에는 무조건 들어가니 if문 밖에 설정
313     bookmarkService.saveBookmarkFromEpisodeBuyDto(dto);
314
315     return ResponseEntity.ok(null);
316 }

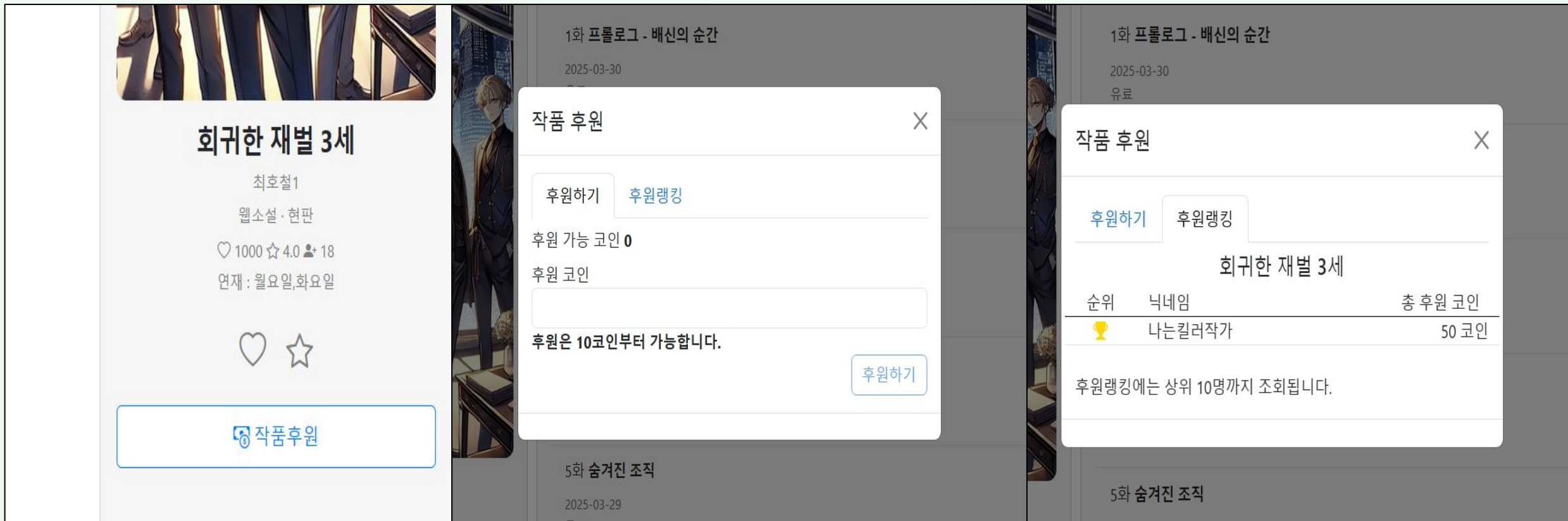
```

```

29 // 글로벌 티켓 중 가장 오래된 데이터의 id 반환
30 @Query(value = "SELECT id FROM TICKETS "
31           + "WHERE user_id = :userId AND grade = :grade "
32           + "ORDER BY created_time ASC LIMIT 1", nativeQuery = true)
33 Long findOldestGlobalTicketId(@Param("userId") Long userId, @Param("grade") int grade);
34
35 // 노벨 티켓중 가장 오래된 데이터의 id 반환
36 @Query(value = "SELECT id FROM TICKETS "
37           + "WHERE user_id = :userId AND novel_id = :novelId AND grade = :grade "
38           + "ORDER BY created_time ASC LIMIT 1", nativeQuery = true)
39 Long findOldestNovelTicketId(@Param("userId") Long userId, @Param("novelId") Long novelId, @Param("grade") int grade);
40

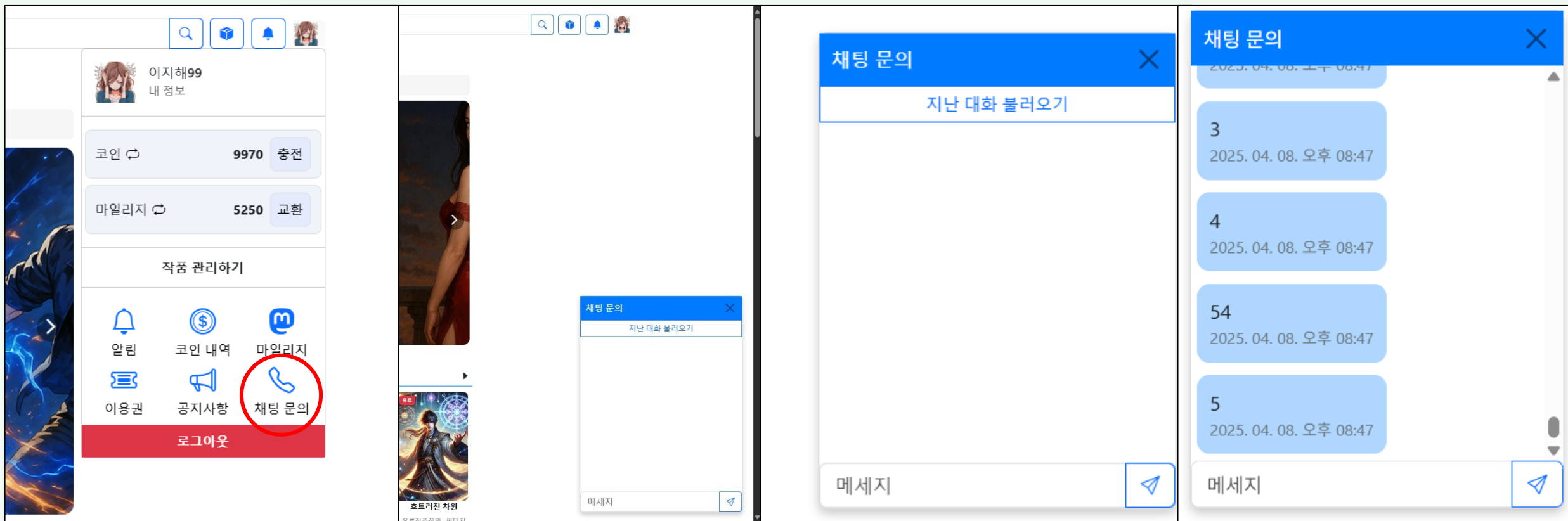
```

- 사용자가 작품을 구매할 경우 구매 방법을 선택 가능.
- 코인으로 구매할 경우 작품의 구매 가격만큼 사용자의 보유 코인이 차감.
- 이용권으로 구매할 경우 가장 오래된 이용권을 먼저 사용.



- 사용자가 작품에 후원을 할 수 있는 시스템.
- 후원 받은 작품의 작가에게 알림 데이터가 생성.

```
17 // 후원한 유저들 랭킹 조회
18 @Query(value = "SELECT u.nickname AS nickname, SUM(cp.coin) AS totalDonation " +
19           "FROM CoinPayment cp " +
20           "JOIN User u ON u.id = cp.userId " +
21           "WHERE cp.type = 3 AND cp.novel.id = :novelId " +
22           "GROUP BY u.nickname " +
23           "ORDER BY totalDonation ASC " +
24           "LIMIT 10")
25 List<Object[]> findDonationRankingByNovelId(@Param("novelId") Long novelId);
```



- 채팅방은 페이지 스크롤 위치에 상관없이 우측하단에 고정.
- 채팅방과 채팅 메시지는 DB에 저장하여 입출력.
- 지난 대화 불러오기 버튼 클릭 시 DB에 저장되어 있는 대화 기록을 출력.

```
7  const chatLink = document.querySelector('#chatLink');
8  chatLink.addEventListener('click', () => {
9    axios.get(`/user/chatroom/check?id=${userId}`).then(response => {
10      const result = response.data;
11
12      if (result == true) {
13        if (chatBox.className.match('d-none')) {
14          chatBox.classList.remove('d-none');
15          const profileDropdown = new bootstrap.Dropdown(document.getElementById('profileDropdown'));
16          profileDropdown.hide();
17        }
18      } else {
19        let result = confirm('1대1 문의할 수 있는 채팅방이 만들어집니다.\n채팅방을 만드시겠습니까?');
20        if (result) {
21          axios.post(`/user/chatroom/create?userId=${userId}&loginId=${loginId}`).then(response => {
22            alert('채팅방이 만들어졌습니다.');
```

stompClient.send("/app/newChatRoom", {}, `\${loginId}님이 새로운 채팅방을 개설했습니다!`);

```
23          }).catch(error => {
24            console.error(error);
25          });
26        }
27      }
28    }).catch(error => {
29      console.error(error);
30    });
31  });
32  });
```

- 채팅방은 '문의하기' 버튼을 클릭하여 생성이 가능하며, 이미 채팅방이 생성되어 있는 상태인 경우 생성하지 않음.


```
45 let socket = new SockJS('/chat');
46 let stompClient = Stomp.over(socket);
47
48 console.log(socket);
49 console.log(stompClient);
50
51 // 연결 설정
52 stompClient.connect({}, function (frame) {
53     console.log("WebSocket 연결 성공:", frame);
54
55     // 구독 예제 (예: "/topic/messages" 경로)
56     stompClient.subscribe('/topic/messages', function (message) {
57         console.log("수신된 메시지:", JSON.parse(message.body));
58         let receivedMessage = JSON.parse(message.body);
59         if (receivedMessage.chatRoomId == userId && receivedMessage.senderLoginId != loginId) {
60             showMessage(receivedMessage);
61         }
62     });
63 });
```

- 클라이언트에서 SockJS와 Stomp.js를 활용하여 WebSocket 서버에 연결.
- 연결 성공 후, 지정된 구독 채널(/topic/message) 에 가입하여 서버로부터 수신되는 메시지를 처리.
- 수신된 메시지의 chatRoomId와 로그인되어 있는 아이디를 비교하여 본인 이외의 상대가 보낸 메시지인 경우에만 화면에 표시.

```

14 @Controller
15 @RequiredArgsConstructor
16 @Slf4j
17 public class ChatWebSocketHandler {
18
19     private final ChatService chatService;
20
21     @MessageMapping("/sendMessage") // "/app/sendMessage"로 오는 메시지를 처리
22     @SendTo("/topic/messages") // "/topic/messages"로 메시지를 전송
23     public ChatMessage sendMessage(Message<ChatMessage> message) {
24         log.info("ChatWebSocketHandler::sendMessage");
25         log.info("message=", message);
26         ChatMessage chatMessage = message.getPayload();
27         chatService.saveMessage(chatMessage); // DB에 메시지 저장
28         return chatMessage; // 채팅 메시지 전송
29     }
30
31     @MessageMapping("/newChatRoom")
32     @SendTo("/topic/admin")
33     public String notifyNewChatRoom(Message<String> message) {
34         log.info("ChatWebSocketHandler::notifyNewChatRoom");
35         return message.getPayload(); // 어드민에게 메시지 전송
36     }
37 }

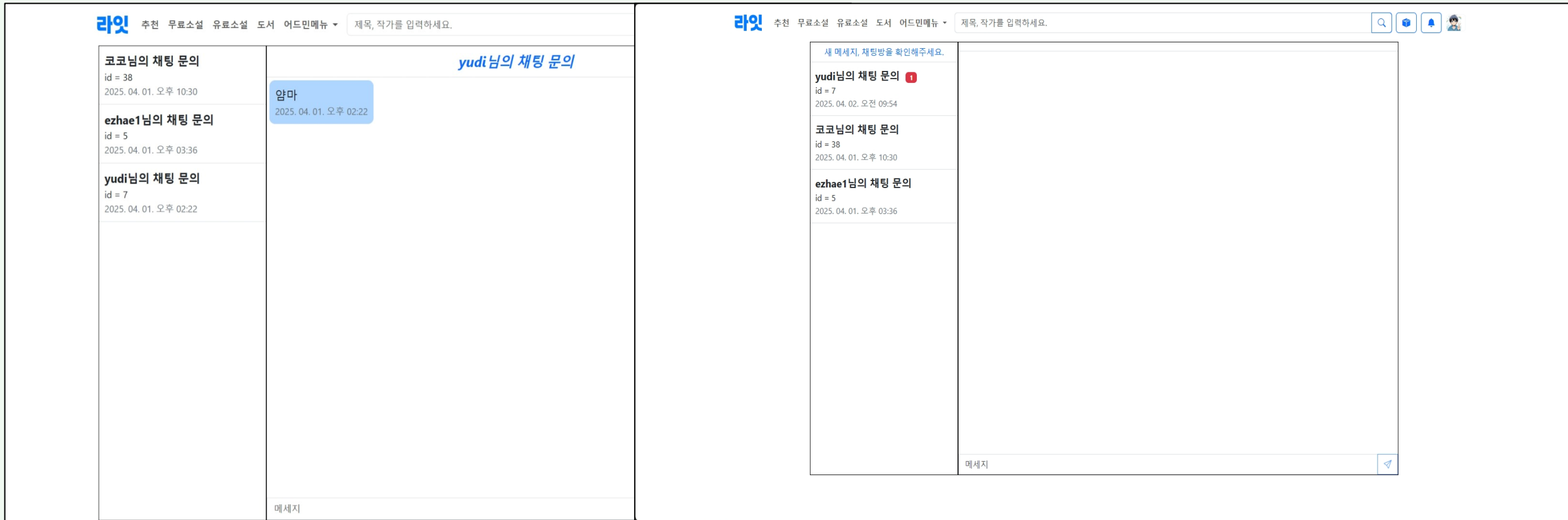
```

- 웹소켓 핸들러를 활용하여 사용자 간 실시간 메시지 송수신을 처리.
- 서버는 수신한 메시지를 데이터베이스에 저장한 뒤, 구독 중인 클라이언트에게 메시지를 전송.
- 클라이언트는 전달받은 메시지를 채팅창에 실시간으로 출력하며, 최신 메시지로 자동 스크롤.

```

65 // 메시지 전송 함수
66 function sendMessage() {
67     let senderId = document.querySelector('span#userId').textContent;
68     let senderLoginId = document.querySelector('span#userLoginId').textContent;
69     const receiverId = 16;
70     const receiverLoginId = 'admin';
71     let chatRoomId = document.querySelector('span#userId').textContent;
72     let content = document.querySelector("input#message").value;
73
74     if (content.trim() !== "") {
75         // 메시지 객체 만들기
76         let message = { senderId, senderLoginId, receiverId, receiverLoginId, chatRoomId, content };
77
78         // 메시지 전송
79         stompClient.send('/app/sendMessage', {}, JSON.stringify(message));
80
81         // 보낸 메시지 채팅창에 추가
82         showMessage(message);
83
84         // 메시지 보내고 입력칸 비우기
85         document.querySelector('input#message').value = "";
86     }
87 }
88
89 // 채팅창에 메시지를 추가하는 함수
90 function showMessage(message) {
91     let chatBox = document.querySelector('div#chat-body'); // 채팅 메시지를 표시할 div
92
93     let sendTime = formatDateTime(Date.now());
94
95     let messageClass = (message.senderLoginId === loginId) ? 'user' : 'admin';
96     let html =
97     `
98         <div class="chatMessageItem ${messageClass}">
99             <div>
100                 <span class="messageContent">${message.content}</span>
101             </div>
102             <div class="text-secondary text-end">
103                 <span>${sendTime}</span>
104             </div>
105         </div>
106     `;
107
108     chatBox.innerHTML += html;
109
110     // 가장 최근 메시지로 스크롤
111     chatBox.scrollTop = chatBox.scrollHeight;
112 }

```



- 어드민은 사용자별 채팅방 목록과 메시지를 실시간으로 확인 및 응답 가능.
- WebSocket을 통해 메시지를 실시간으로 송수신하며, 읽지 않은 메시지는 뱃지로 표시.
- 채팅방 클릭 시 해당 메시지를 불러오고, 관리자와 사용자 메시지를 시각적으로 구분해 출력.
- 채팅방 렌더링 함수를 호출할 때 안 읽은 메시지의 렌더링의 순서는 받은 메시지 시간을 우선으로 하며, 같은 경우 안 읽은 메시지 개수로 정렬.

```
138 let socket = new SockJS('/chat');
139 let stompClient = Stomp.over(socket);
140
141 console.log(socket);
142 console.log(stompClient);
143
144 // 연결 설정
145 stompClient.connect({}, function (frame) {
146     console.log("WebSocket 연결 성공:", frame);
147
148     // 구독 예제 (예: "/topic/messages" 경로)
149     stompClient.subscribe('/topic/messages', function (message) {
150         console.log("수신된 메시지:", JSON.parse(message.body));
151         let receivedMessage = JSON.parse(message.body);
152         let chatDatas = document.querySelector('div#chatData');
153         if (receivedMessage.senderLoginId == chatDatas.dataset.sendloginid) {
154             showMessage(receivedMessage);
155         }
156         reloadChatRoom();
157     });
158
159     stompClient.subscribe('/topic/admin', function (message) {
160         reloadChatRoom();
161     });
162 });
```

- 클라이언트에서 SockJS와 Stomp.js를 활용하여 WebSocket 서버에 연결.
- 연결 성공 후, 지정된 구독 채널(/topic/message)에 가입하여 서버로부터 수신되는 메시지를 처리.
- 메시지 수신 시, 현재 열린 채팅방이면 바로 표시하고, 아니라면 새 메시지 알림 버튼을 활성화.

```

14 @Controller
15 @RequiredArgsConstructor
16 @Slf4j
17 public class ChatWebSocketHandler {
18
19     private final ChatService chatService;
20
21     @MessageMapping("/sendMessage") // "/app/sendMessage"로 오는 메시지를 처리
22     @SendTo("/topic/messages") // "/topic/messages"로 메시지를 전송
23     public ChatMessage sendMessage(Message<ChatMessage> message) {
24         log.info("ChatWebSocketHandler::sendMessage");
25         log.info("message=", message);
26         ChatMessage chatMessage = message.getPayload();
27         chatService.saveMessage(chatMessage); // DB에 메시지 저장
28         return chatMessage; // 채팅 메시지 전송
29     }
30
31     @MessageMapping("/newChatRoom")
32     @SendTo("/topic/admin")
33     public String notifyNewChatRoom(Message<String> message) {
34         log.info("ChatWebSocketHandler::notifyNewChatRoom");
35         return message.getPayload(); // 어드민에게 메시지 전송
36     }
37 }

```

- 웹소켓 핸들러를 활용하여 사용자 간 실시간 메시지 송수신을 처리.
- 서버는 수신한 메시지를 데이터베이스에 저장한 뒤, 구독 중인 클라이언트에게 메시지를 전송.
- 클라이언트는 전달받은 메시지를 채팅창에 실시간으로 출력하며, 최신 메시지로 자동 스크롤.

```

164 // 메시지 전송 함수
165 function sendMessage() {
166     let chatData = document.querySelector('div#chatData');
167     console.log(chatData);
168     let senderId = 16;
169     let senderLoginId = 'admin';
170     let receiverId = chatData.dataset.sendid;
171     let receiverLoginId = chatData.dataset.sendloginid;
172     let chatRoomId = chatData.dataset.chatroomid;
173     let content = document.querySelector("input#admin-message").value;
174     let isRead = 0;
175
176     if (content.trim() !== "") {
177         // 메시지 객체 만들기
178         let message = { senderId, senderLoginId, receiverId, receiverLoginId, chatRoomId, content, isRead };
179
180         // 메시지 전송
181         stompClient.send('/app/sendMessage', {}, JSON.stringify(message));
182
183         // 보낸 메시지 채팅창에 추가
184         showMessage(message);
185
186         // 메시지 보내고 입력칸 비우기
187         document.querySelector('input#admin-message').value = "";
188     }
189 }
190
191 // 채팅창에 메시지를 추가하는 함수
192 function showMessage(message) {
193     let chatBox = document.querySelector('div#admin-chat-body'); // 채팅 메시지를 표시할 div
194
195     let sendTime = formatDate(Date.now());
196
197     let messageClass = (message.senderLoginId === 'admin') ? 'admin' : 'user';
198     let html =
199     `
200         <div class="adminChatMessageItem ${messageClass}">
201             <div>
202                 <span class="fs-5">${message.content}</span>
203             </div>
204             <div class="text-secondary text-end">
205                 <span>${sendTime}</span>
206             </div>
207         </div>
208     `;
209
210     chatBox.innerHTML += html;
211
212     // 가장 최근 메시지로 스크롤
213     chatBox.scrollTop = chatBox.scrollHeight;
214 }

```

```
216 const btnReloadRoom = document.querySelector('button#btnReloadRoom');
217 // 새 메세지 및 채팅방 알림버튼
218 function reloadChatRoom() {
219   if (btnReloadRoom.className.match('d-none')) {
220     btnReloadRoom.classList.remove('d-none');
221   }
222 }
223 btnReloadRoom.addEventListener('click', () => {
224   document.querySelector('div#adminChatBox').innerHTML =
225     `
226       <div class="spinner-border text-center" role="status">
227         <span class="visually-hidden"></span>
228       </div>
229       <span>채팅방 불러오는중...</span>
230     `;
231   getChatRoom();
232   btnReloadRoom.classList.add('d-none');
233 });
```

- 새로운 채팅방이 생성되거나 메시지가 수신되면 "새로고침" 버튼이 표시되어, 관리자가 목록을 수동으로 갱신.

